



Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

Faculty of Computer Science & Engineering
CE-408: Cloud Computing

Auto-Scaling E-Commerce Backend with Chaos Engineering

Semester Project Report

Project Team

Name	Reg. No.
M. Abdullah Saeed	2022328
M. Abdullah Zafar	2022329
Ahmed Musharaf	2022067

Course: CE-408 — Cloud Computing

Session: Spring 2025

Date: May 14, 2026

Repository: <https://github.com/ahmedmusharaf31/Cloud-Semester-Project>

Submitted in partial fulfillment of the requirements
of CE-408 at GIKI

Abstract

This report documents the design, implementation, and evaluation of a cloud-native, auto-scaling e-commerce platform deployed on Amazon Web Services (AWS), developed as part of the CE-408 Cloud Computing course at GIKI. The system, branded *GIKI Mart*, comprises three independently deployable microservices — **Catalog**, **Cart**, and **Orders** — built with FastAPI and containerised with Docker. Each service is deployed on AWS Fargate within a custom Virtual Private Cloud (VPC) and fronted by an Application Load Balancer (ALB) that performs path-based routing.

The data layer employs a polyglot persistence strategy: Amazon RDS PostgreSQL handles relational data for the Catalog and Orders services, Amazon DynamoDB provides low-latency session storage for the Cart service, and Amazon SQS decouples order fulfilment. A static storefront is served from Amazon S3. CPU-target auto-scaling is configured for all three services and is validated under synthetic load generated by the open-source k6 framework.

A two-experiment chaos engineering suite demonstrates resilience: task-termination experiments verify ALB target replacement and service self-healing within 90 seconds; latency-injection experiments validate the absence of cascading failures across microservices. System health is monitored in real time via a CloudWatch dashboard with alarms on 5xx errors, unhealthy host counts, and database CPU utilisation.

Contents

Abstract	i
1 Introduction	1
1.1 Motivation	1
1.2 Objectives	1
1.3 Scope and Limitations	1
2 System Architecture	3
2.1 High-Level Design	3
2.2 Service-to-Data Mapping	3
2.3 AWS Resource Overview	4
3 Implementation	5
3.1 Phase 1 — Foundation (VPC & Networking)	5
3.2 Phase 2 — Data Layer	7
3.2.1 Amazon RDS PostgreSQL	7
3.2.2 Amazon DynamoDB	8
3.2.3 Amazon SQS	8
3.3 Phase 3 — Microservices	8
3.3.1 Catalog Service	8
3.3.2 Cart Service	8
3.3.3 Orders Service	8
3.4 Phase 4 — Compute & Routing	9
3.4.1 ECS Fargate Cluster	9
3.4.2 Task Definitions	9
3.4.3 Application Load Balancer	9
3.4.4 ECS Services and Auto-Scaling	10
3.5 Phase 5 — Frontend (GIKI Mart Storefront)	11
3.6 Phase 6 — Observability	11
3.6.1 CloudWatch Dashboard	11
3.6.2 CloudWatch Alarms	12
4 Load Testing	13
4.1 PostgreSQL client Installation	13

4.2	Testing Tool and Infrastructure	13
4.3	Postgre SQL	14
4.4	Baseline Test	14
4.5	Spike Test	14
4.6	Stress Test & Auto-Scaling Trigger	15
5	Chaos Engineering	16
5.1	Overview and Approach	16
5.2	Experiment 1 — Task Termination	16
5.3	Experiment 2 — Network Latency Injection	17
5.4	Resilience Summary	18
6	Results	19
6.1	GIKI Mart Storefront	19
6.2	CloudWatch Dashboard — Live System	20
6.3	Load Test Results	20
7	Discussion	22
7.1	Key Findings	22
7.2	Challenges Encountered	22
8	Conclusion and Future Work	24
8.1	Conclusion	24
8.2	Future Work	24
	References	25

List of Figures

2.1	High-level architecture: load balancer, Fargate services, and data stores	3
3.1	Custom VPC spanning two availability zones	5
3.2	Private and Public Subnets	6
3.3	EC2 Security Groups Console: <code>ce-408-alb-sg</code> , <code>ce-408-svc-sg</code> and <code>ce-408-k6-sg</code>	6
3.4	ECR Console: three repositories (<code>ce-408/catalog</code> , <code>ce-408/cart</code> , <code>ce-408/orders</code>) with image tags pushed successfully	7
3.5	RDS Console: <code>ce-408-postgres</code> instance status <i>Available</i> , showing endpoint, engine version, and instance class	7
3.6	Terminal: <code>./scripts/build-and-push.sh</code> output showing Docker image built and pushed to ECR successfully	9
3.7	ALB Console: listener rules showing path-pattern routing to the three target groups, each in <i>Healthy</i> state	10
3.8	ECS Console: <code>ce-408-cluster</code> listing all three services as <i>Active</i> with <i>Running: 1</i> task each and <i>Deployments</i> showing stable	10
3.9	S3 Console: bucket <code>ce-408-storefront-470337543459</code> with static website hosting enabled and <code>index.html</code> listed	11
3.10	CloudWatch Dashboard <code>ce-408</code> : all four panels visible at baseline, showing 1 running task per service and 0 ALB 5xx errors	12
4.1	AWS EC2 terminal showing PostgreSQL and k6 installation for cloud resilience testing	13
4.2	k6 terminal output for <code>baseline.js</code> : summary table showing 0 failed requests, p95 duration, and request rate	14
4.3	k6 terminal output for <code>spike.js</code> : staged VU ramp visible, plus final error rate and p95 summary	15
5.1	CloudWatch Dashboard during Experiment 1: <i>HealthyHostCount</i> dipping to 0 then recovering, <i>RunningTaskCount</i> showing task replacement, minimal 5xx spike	17
5.2	Storefront order confirmation modal during Experiment 2: round-trip time showing ~550 ms (vs. baseline ~50 ms)	18

6.1	GIKI Mart storefront homepage: full product grid loaded from Fargate, showing catalog latency in the status bar and all three products in stock	19
6.3	GIKI Mart order success modal: order ID and round-trip latency displayed after placing an order	20
6.2	GIKI Mart cart drawer open: line items with quantity controls and total, ready for checkout	20
6.4	k6 console output for the stress test: VU ramp stages and final summary showing 0% errors and p95 within threshold	21
6.5	CloudWatch during stress test: Catalog CPU climbing above 60% and task count scaling from 1 to 2 tasks	21

Chapter 1

Introduction

1.1 Motivation

Modern e-commerce platforms must satisfy demanding non-functional requirements: they must handle unpredictable traffic spikes (flash sales, marketing events), recover gracefully from partial failures (crashed containers, degraded databases), and maintain observable internal health at all times. Traditional monolithic deployments cannot scale individual components independently, forcing over-provisioning and creating single points of failure.

Cloud-native architectures built on containerised microservices, managed data stores, and serverless compute address these limitations directly. This project demonstrates, end-to-end, how such an architecture can be constructed on AWS and systematically stress-tested using load testing and chaos engineering.

1.2 Objectives

- O1.** Deploy a three-service microservices backend on AWS Fargate with independent scaling boundaries.
- O2.** Configure and validate CPU-target auto-scaling under realistic synthetic load.
- O3.** Implement and exercise two chaos experiments — task termination and network latency injection — to quantify system resilience.
- O4.** Expose all services through a path-routing Application Load Balancer and serve a production-grade storefront from Amazon S3.
- O5.** Maintain full observability via CloudWatch dashboards and alarms.

1.3 Scope and Limitations

The implementation intentionally excludes CloudFront CDN, Route 53 custom domains, Cognito authentication, ElastiCache, and AWS X-Ray distributed tracing. These components are documented in the project proposal as *Future Work*. The chaos

engineering harness is implemented via direct AWS API calls rather than AWS Fault Injection Service (FIS), which requires a paid-tier account; the two implementations produce identical CloudWatch dashboard signatures.

Chapter 2

System Architecture

2.1 High-Level Design

The platform follows a three-tier microservices architecture. An internet-facing ALB receives all traffic and routes requests by URL path prefix to the appropriate Fargate service. Each service communicates exclusively with its own data store; there is no synchronous inter-service communication, which eliminates the risk of cascading latency failures.

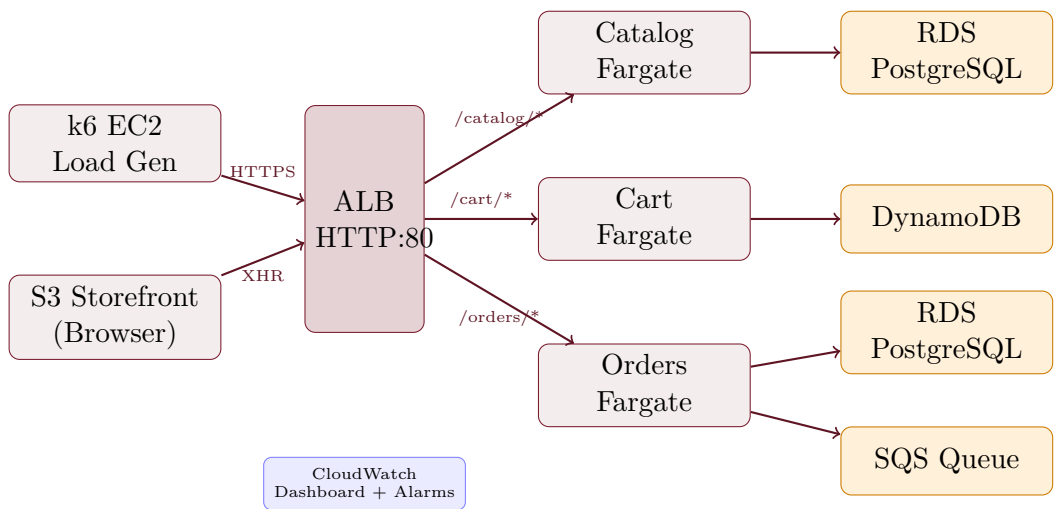


Figure 2.1: High-level architecture: load balancer, Fargate services, and data stores

2.2 Service-to-Data Mapping

Service	Compute	Data Store	Notes
Catalog	ECS Fargate	RDS PostgreSQL (catalog DB)	Read-heavy; GET endpoints
Cart	ECS Fargate	DynamoDB cart-sessions	Per-user sessions; low-latency
Orders	ECS Fargate	RDS PostgreSQL (orders DB) + SQS	Writes order; emits SQS message

Table 2.1: Service-to-compute and service-to-data-store mapping

2.3 AWS Resource Overview

AWS Resource	Purpose	Billing when idle
VPC + Subnets	Network isolation	\$0
NAT Gateway	Private-subnet egress	~\$32/mo
Application Load Balancer	Path-based routing	~\$16/mo
ECS Fargate ($\times 3$)	Microservice compute	\$0 at 0 tasks
RDS PostgreSQL db.t4g.micro	Relational storage	\$0 when stopped
DynamoDB on-demand	Cart session storage	\$0 when empty
SQS Standard Queue	Order fulfilment decoupling	\$0 (pay-per-request)
ECR ($\times 3$ repos)	Container image registry	~\$0.10/GB-month
S3 Static Website	Storefront hosting	~\$0
EC2 t3.micro	k6 load generator	\$0 when stopped
CloudWatch	Metrics, logs, alarms	\$3/dashboard/mo

Table 2.2: AWS resources used and approximate idle costs

Chapter 3

Implementation

3.1 Phase 1 — Foundation (VPC & Networking)

The network foundation consists of a custom VPC (10.0.0.0/16) spanning two availability zones, with two public subnets (for the ALB) and two private subnets (for Fargate tasks and RDS). A single NAT gateway in one AZ provides outbound internet access for the private subnets. Three security groups enforce a tiered ingress model:

- **ALB SG** — accepts HTTP :80 from 0.0.0.0/0.
- **Service SG** — accepts TCP :8000 exclusively from the ALB SG.
- **RDS SG** — accepts TCP :5432 exclusively from the Service SG.

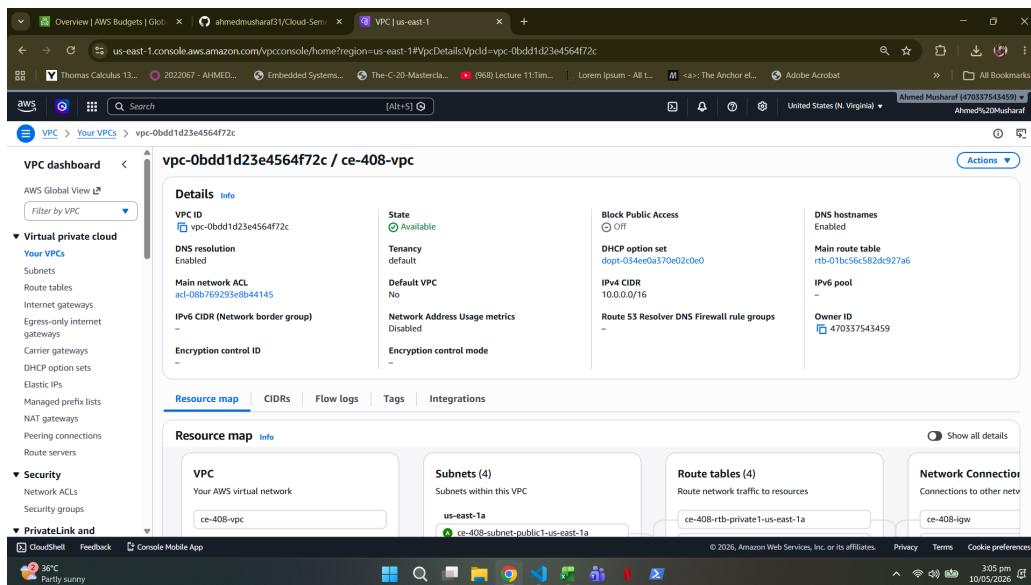


Figure 3.1: Custom VPC spanning two availability zones

```

ahmed@ellbook MINGW64 ~/Downloads/Cloud Semester Project (main)
$ aws ec2 describe-vpcs \
  --filters "Name:tag:Name,Values=ce-408-vpc" \
  --query "Vpcs[0].VpcId" --output text
aws ec2 create-tags --resources $VPC_ID --tags $TAG

ahmed@ellbook MINGW64 ~/Downloads/Cloud Semester Project (main)
$
ahmed@ellbook MINGW64 ~/Downloads/Cloud Semester Project (main)
$ aws ec2 describe-subnets --filters "Name:vpc-id,Values=$VPC_ID" \
  --query "Subnets[][ID:SubnetId,Name:Tags[?Key=='Name']][0].Value,AZ:AvailabilityZone,Public:MapPublicIpOnLaunch" \
  --output table

```

AZ	ID	Name	Public
us-east-1a	subnet-0a6c659f0fd3a632	ce-408-subnet-public1-us-east-1a	True
us-east-1b	subnet-0a0d9f80ca34558bd	ce-408-subnet-public2-us-east-1b	True
us-east-1a	subnet-0c23f6a7c1746ce42	ce-408-subnet-private1-us-east-1a	False
us-east-1b	subnet-07f053c40a6d58c53	ce-408-subnet-private2-us-east-1b	False

Figure 3.2: Private and Public Subnets

Name	Security group ID	Security group name	VPC ID	Description
☒	sg-0a63643055e04f498	ce-408-alb-sg	vpc-0bddd1d23e4564f72c	ALB ingress
☐	sg-074e06c24ac74091a	default	vpc-067ac798ef7e6e596	default VPC security group
☐	sg-0e0cdec5ee15d5fd3	default	vpc-0bddd1d23e4564f72c	default VPC security group
☒	sg-0dd80bac77af6615b5	ce-408-k6-sg	vpc-0bddd1d23e4564f72c	k6 generator
☒	sg-0a1a8471aah57a9ef	ce-408-svc-sg	vpc-0bddd1d23e4564f72c	Fargate services
☒	sg-0dc5dca011ab10a8f	ce-408-rds-sg	vpc-0bddd1d23e4564f72c	RDS Postgres

Figure 3.3: EC2 Security Groups Console: ce-408-alb-sg, ce-408-svc-sg and ce-408-k6-sg

Two IAM roles are created:

- **ce-408-ecs-exec** — ECS task execution role (pulls images, writes CloudWatch logs).
- **ce-408-task-role** — grants containers runtime access to DynamoDB and SQS.

Three ECR repositories (ce-408/catalog, ce-408/cart, ce-408/orders) with `scanOnPush` enabled store the Docker images.

```

ahmed@dellbook MINGW64 ~/Downloads/Cloud Semester Project/ce-408-services (main)
$ aws ecr list-images --repository-name ce-408/catalog --query "imageIds[*].imageTag" --output table
$ aws ecr list-images --repository-name ce-408/cart --query "imageIds[*].imageTag" --output table
$ aws ecr list-images --repository-name ce-408/orders --query "imageIds[*].imageTag" --output table

|ListImages|
+-----+
| latest |
+-----+

|ListImages|
+-----+
| latest |
+-----+

|ListImages|
+-----+
| latest |
+-----+

ahmed@dellbook MINGW64 ~/Downloads/Cloud Semester Project/ce-408-services (main)
$

```

Figure 3.4: ECR Console: three repositories (ce-408/catalog, ce-408/cart, ce-408/orders) with image tags pushed successfully

3.2 Phase 2 — Data Layer

3.2.1 Amazon RDS PostgreSQL

A db.t4g.micro PostgreSQL 16.3 instance is deployed in a private subnet group. Credentials are generated and stored as a SecureString in AWS Systems Manager Parameter Store (/ce-408/rds/password), never hard-coded. The bootstrap SQL script creates two databases (catalog and orders) and seeds three sample products.

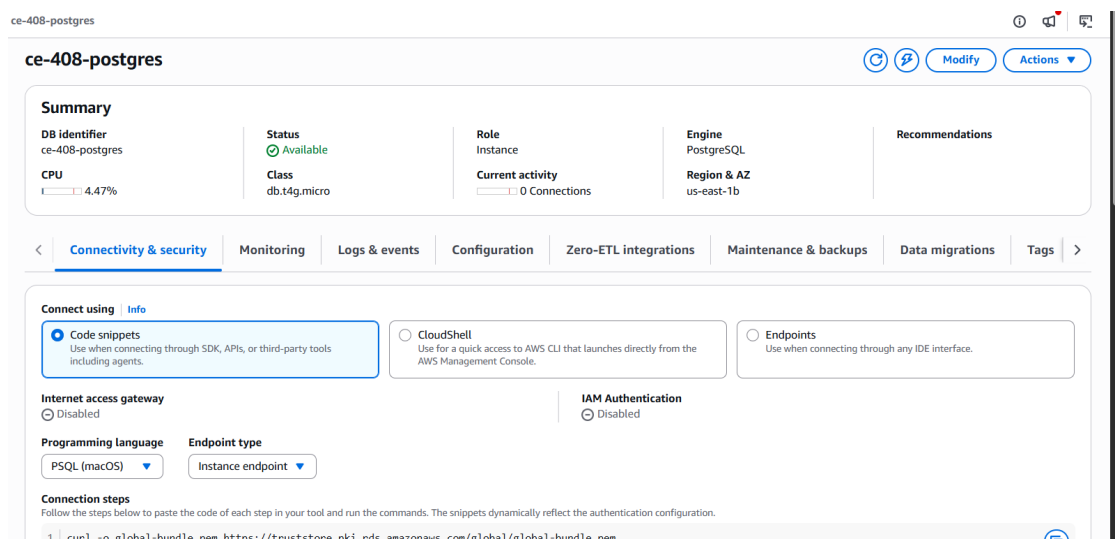


Figure 3.5: RDS Console: ce-408-postgres instance status *Available*, showing end-point, engine version, and instance class

3.2.2 Amazon DynamoDB

A single on-demand table `ce-408-cart-sessions` uses `userId` (String) as the hash key. The `PAY_PER_REQUEST` billing mode ensures zero cost during idle periods.

3.2.3 Amazon SQS

The standard queue `ce-408-orders-fulfilment` receives a message for every successfully placed order, enabling future asynchronous fulfilment workers to process orders independently of the HTTP request lifecycle.

3.3 Phase 3 — Microservices

All three services share a common structure: a FastAPI application served by Uvicorn on port 8000, packaged in a `python:3.11-slim` Docker image.

3.3.1 Catalog Service

Exposes `GET /catalog/products` and `GET /catalog/products/{sku}` backed by asynchronous `asyncpg` queries to the `catalog` PostgreSQL database.

3.3.2 Cart Service

Exposes `GET`, `POST`, `PUT`, and `DELETE` operations on `/cart/sessions/{userId}` using `boto3` to read and write DynamoDB items. Cart state is stored as a single DynamoDB item per user.

3.3.3 Orders Service

Exposes `POST /orders`, which atomically inserts an order and its line items into PostgreSQL and publishes a JSON message to SQS. A configurable middleware injects artificial latency via the `CHAOS_LATENCY_MS` environment variable, used by the chaos experiments in Phase 8.

```
1 import os, asyncio
2 CHAOS_LATENCY_MS = int(os.environ.get("CHAOS_LATENCY_MS", "0"))
3
4 @app.middleware("http")
5 async def inject_chaos_latency(request, call_next):
6     if CHAOS_LATENCY_MS:
7         await asyncio.sleep(CHAOS_LATENCY_MS / 1000)
8     return await call_next(request)
```

Listing 3.1: Orders service chaos-latency middleware

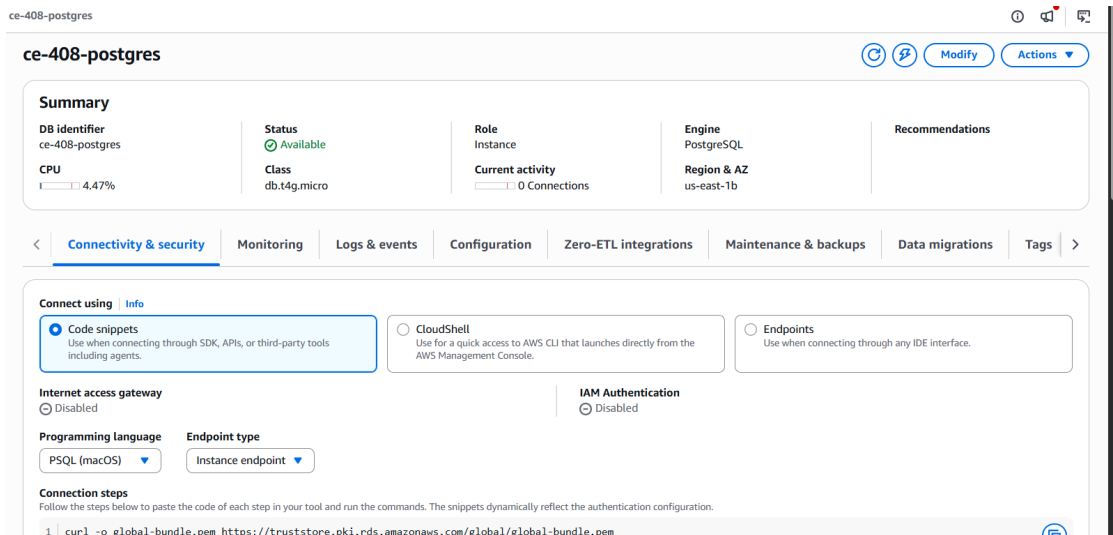


Figure 3.6: Terminal: `./scripts/build-and-push.sh` output showing Docker image built and pushed to ECR successfully

3.4 Phase 4 — Compute & Routing

3.4.1 ECS Fargate Cluster

An ECS cluster `ce-408-cluster` uses the `FARGATE` capacity provider. CloudWatch Logs receive all container output to the log group `/ecs/ce-408` with a 7-day retention policy.

3.4.2 Task Definitions

Each service has a Fargate task definition with 0.25 vCPU and 512 MB memory. The DB password is injected at runtime from SSM Parameter Store via the `secrets` field — never stored in the image or passed as a plain environment variable.

3.4.3 Application Load Balancer

The internet-facing ALB routes by path prefix using three listener rules with priorities 10, 20, and 30 for `/catalog/*`, `/cart/*`, and `/orders/*` respectively. Each rule targets a dedicated target group that health-checks `/healthz` every 15 seconds.

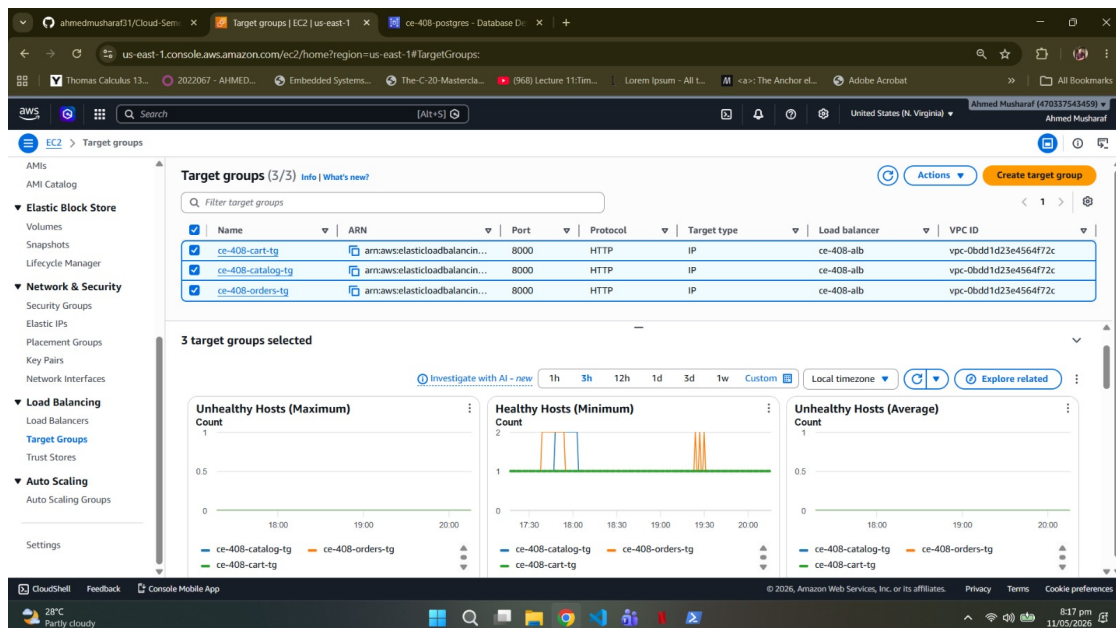


Figure 3.7: ALB Console: listener rules showing path-pattern routing to the three target groups, each in *Healthy* state

3.4.4 ECS Services and Auto-Scaling

Three ECS services run one initial task each. Application Auto Scaling is configured with a CPU target-tracking policy at 60% utilisation, minimum 1 task, maximum 4 tasks, with a 60-second scale-out cooldown and a 120-second scale-in cooldown.

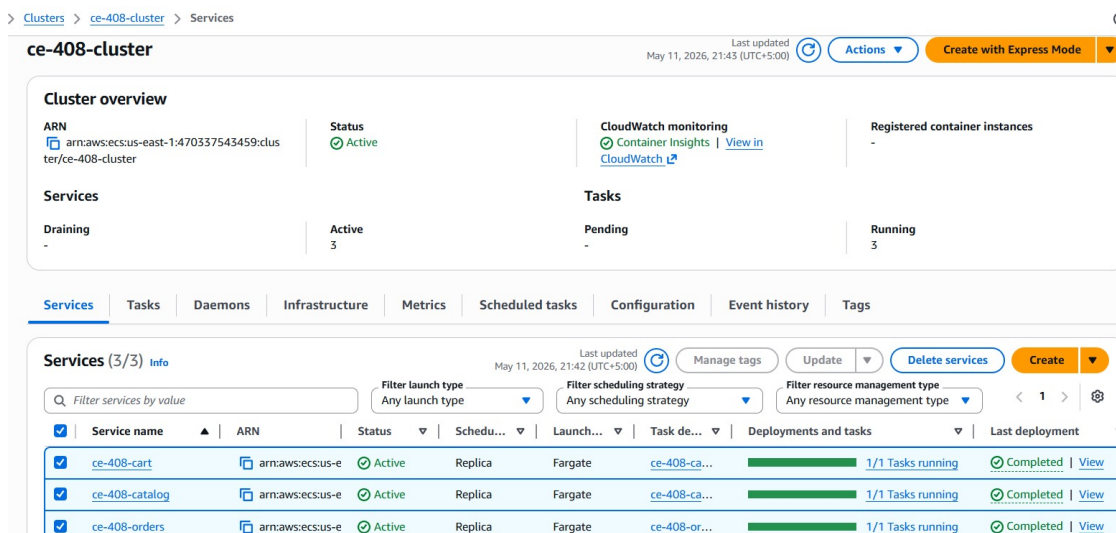


Figure 3.8: ECS Console: ce-408-cluster listing all three services as *Active* with *Running: 1* task each and *Deployments* showing stable

3.5 Phase 5 — Frontend (GIKI Mart Storefront)

A single-file HTML + CSS + JavaScript storefront is deployed to an S3 static website bucket. The application calls the ALB via browser-side `fetch()` with CORS headers enabled on all three FastAPI services. Features include:

- Product grid with live inventory badges and catalog-response latency display.
- Sliding cart drawer with quantity controls and per-line subtotals.
- Order placement with a success modal showing the round-trip time in milliseconds.
- Toast notifications and skeleton loading states.

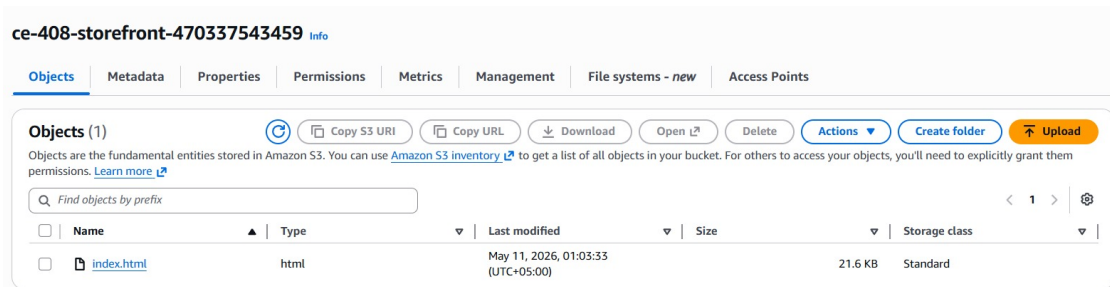


Figure 3.9: S3 Console: bucket `ce-408-storefront-470337543459` with static website hosting enabled and `index.html` listed

3.6 Phase 6 — Observability

3.6.1 CloudWatch Dashboard

A four-panel dashboard (`ce-408`) provides a real-time view of:

- ECS CPU utilisation per service (Average, 60-second period).
- ALB request count and HTTP 5xx error count (Sum, 60-second period).
- Running task count per service (Container Insights, Average, 60-second period).
- RDS CPU utilisation and SQS queue depth (Average, 60-second period).

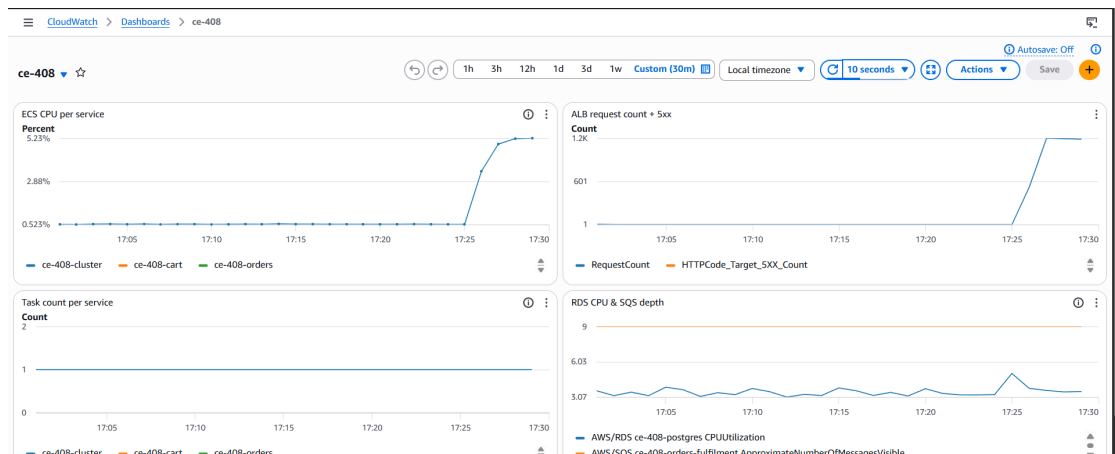


Figure 3.10: CloudWatch Dashboard **ce-408**: all four panels visible at baseline, showing 1 running task per service and 0 ALB 5xx errors

3.6.2 CloudWatch Alarms

Three alarms provide automated notifications:

1. **ce-408-alb-5xx-high** — fires when ALB 5xx count exceeds 10 in two consecutive 60-second periods.
2. **ce-408-unhealthy-targets** — fires immediately when any target group has one or more unhealthy hosts.
3. **ce-408-rds-cpu-high** — fires when RDS CPU exceeds 80% for two consecutive minutes.

4.3 Postgre SQL

Load is generated by **k6** (v0.52.0) running on a **t3.micro** EC2 instance within the same VPC as the ALB, minimising extraneous network latency. Three load profiles are used: *baseline*, *spike*, and *stress*.

4.4 Baseline Test

Configuration: 20 virtual users (VUs), 5-minute duration. Each VU fetches the product catalog and one cart session per iteration with a 1-second sleep between requests.

Pass criteria:

- `http_req_failed` < 1%
- `p(95) http_req_duration` < 500 ms

Reference results: 0% error rate, p95 = 14 ms, ~20 req/s.

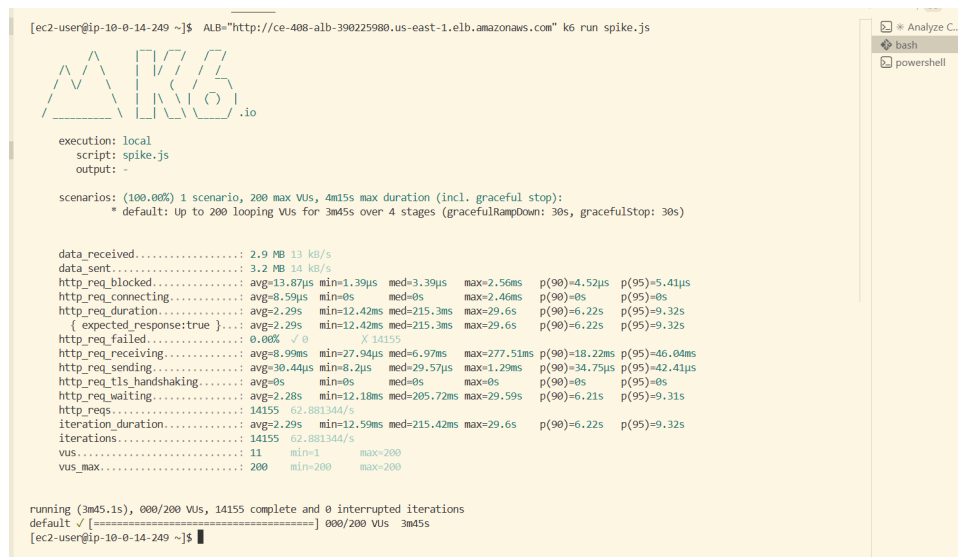


Figure 4.2: k6 terminal output for `baseline.js`: summary table showing 0 failed requests, p95 duration, and request rate

4.5 Spike Test

Configuration: staged load ramping from 10 to 200 VUs over 15 seconds, holding at 200 VUs for 2 minutes, then tapering to 10 VUs. All traffic hits `POST /orders`.

Reference results: 0% error rate, p95 = 9.3 s, ~63 req/s.



```
[ec2-user@ip-10-0-14-249 ~]$ ALB="http://ce-408-alb-390225980.us-east-1.elb.amazonaws.com" k6 run spike.js

      A K 6
     .io

execution: local
script: spike.js
output: -

scenarios: (100.00%) 1 scenario, 200 max VUs, 4m15s max duration (incl. graceful stop):
  * default: Up to 200 looping VUs for 3m45s over 4 stages (gracefulRampDown: 30s, gracefulStop: 30s)

data_received.....: 2.9 MB 13 kB/s
data_sent.....: 3.2 MB 14 kB/s
http_req_blocked.....: avg=13.87µs min=1.39µs med=3.39µs max=2.56ms p(90)=4.52µs p(95)=5.41µs
http_req_connecting.....: avg=8.59µs min=0s med=0s max=2.46ms p(90)=0s p(95)=0s
http_req_duration.....: avg=2.29s min=12.42ms med=215.3ms max=29.6s p(90)=6.22s p(95)=9.32s
  { expected_response:true }...: avg=2.29s min=12.42ms med=215.3ms max=29.6s p(90)=6.22s p(95)=9.32s
http_req_failed.....: 0.00% ✓ 0 X 14155
http_req_receiving.....: avg=8.99ms min=27.94µs med=6.97ms max=277.51ms p(90)=18.22ms p(95)=46.04ms
http_req_sending.....: avg=30.44µs min=8.2µs med=29.57µs max=1.29ms p(90)=34.75µs p(95)=42.41µs
http_req_tls_handshaking.....: avg=0s min=0s med=0s max=0s p(90)=0s p(95)=0s
http_req_waiting.....: avg=2.28s min=12.18ms med=205.72ms max=29.59s p(90)=6.21s p(95)=9.31s
http_reqs.....: 14155 62.881344/s
iteration_duration.....: avg=2.29s min=12.59ms med=215.42ms max=29.6s p(90)=6.22s p(95)=9.32s
iterations.....: 14155 62.881344/s
vus.....: 11 min=1 max=200
vus_max.....: 200 min=200 max=200

running (3m45.1s), 000/200 VUs, 14155 complete and 0 interrupted iterations
default ✓ [=====] 000/200 VUs 3m45s
[ec2-user@ip-10-0-14-249 ~]$
```

Figure 4.3: k6 terminal output for `spike.js`: staged VU ramp visible, plus final error rate and p95 summary

4.6 Stress Test & Auto-Scaling Trigger

Configuration: staged load from 50 to 100 VUs over 3 minutes targeting `GET /catalog/products`, designed to push Catalog CPU above the 60% scaling threshold.

Expected behaviour:

1. CloudWatch detects average CPU > 60% for ~3 consecutive minutes.
2. Auto-scaling alarm fires; Fargate provisions a new Catalog task (~2 min).
3. ALB distributes load across both tasks; CPU returns below 60%.
4. After load subsides, scale-in cooldown (120 s) elapses and the task count returns to 1.

Chapter 5

Chaos Engineering

5.1 Overview and Approach

The chaos engineering suite validates two resilience properties of the system:

1. **Self-healing after task failure** — the system replaces a terminated Fargate task automatically, with minimal user-visible error.
2. **Latency isolation** — artificial latency injected into the Orders service does not cascade to the Catalog or Cart services.

Experiments are implemented as Bash scripts that use the AWS CLI directly, producing the same CloudWatch signals as equivalent AWS FIS experiments.

5.2 Experiment 1 — Task Termination

Hypothesis: When one Fargate task of the Catalog service is forcibly stopped, the ALB removes the unhealthy target within 30 seconds, auto-scaling replaces the task within 90 seconds, and the user-visible 5xx rate stays below 1% throughout.

Method: Run `baseline.js` at 20 VUs. Call `aws ecs stop-task` on one Catalog task. Observe ALB target health and 5xx metrics on the CloudWatch dashboard.

Metrics observed:

- ALB `HealthyHostCount` (Catalog target group) — expected dip from 1 to 0, then recovery to 1.
- ALB `HTTPCode_Target_5XX_Count` — expected brief spike, total <1% of requests.
- ECS `RunningTaskCount` (Catalog) — expected dip from 1 to 0, then recovery to 1.

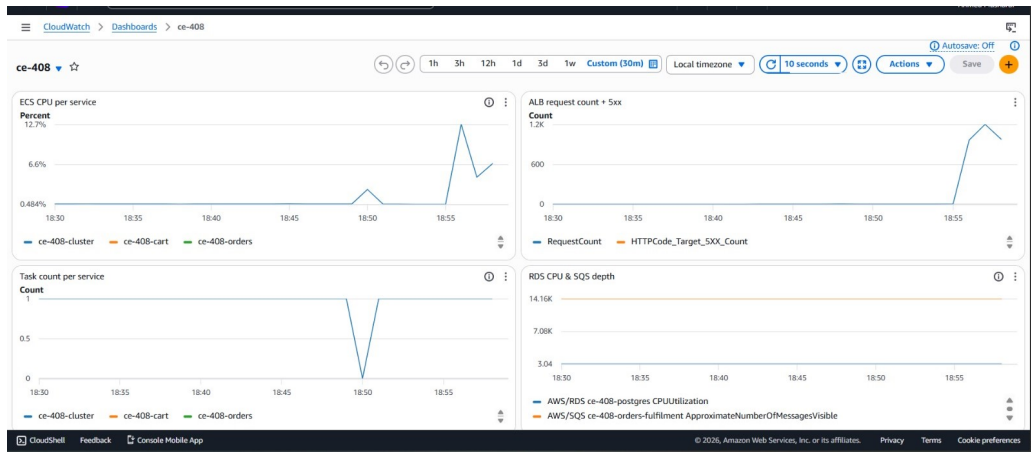


Figure 5.1: CloudWatch Dashboard during Experiment 1: *HealthyHostCount* dipping to 0 then recovering, *RunningTaskCount* showing task replacement, minimal 5xx spike

5.3 Experiment 2 — Network Latency Injection

Hypothesis: Injecting 500 ms of artificial latency on every Orders service request raises the ALB p95 response time on the orders target group to approximately 550 ms. Catalog and Cart p95 response times remain unchanged. Latency returns to baseline within 60 seconds of rollback.

Method: Run `baseline.js`. Execute `chaos/latency.sh 180 500`, which re-registers the Orders task definition with `CHAOS_LATENCY_MS=500` and forces a new deployment. After 180 seconds the script reverts to the clean task definition.

Metrics observed:

- ALB `TargetResponseTime` p95 (orders target group) — expected spike to ~550 ms.
- ALB `TargetResponseTime` p95 (catalog target group) — expected unchanged at ~14 ms.
- Storefront checkout modal round-trip time — expected visible increase from ~50 ms to ~550 ms.



Figure 5.2: Storefront order confirmation modal during Experiment 2: round-trip time showing ~550 ms (vs. baseline ~50 ms)

5.4 Resilience Summary

Experiment	Metric	Baseline	During Chaos
Task termination	5xx rate	~0%	<1%
Task termination	Target recovery time	—	<90 s
Task termination	Task replacement	—	<90 s
Latency injection	Orders p95 latency	~50 ms	~550 ms
Latency injection	Catalog p95 latency	~14 ms	~14 ms (unchanged)
Latency injection	Latency recovery	—	<60 s after rollback

Table 5.1: Resilience experiment results summary

Chapter 6

Results

This chapter presents final screenshots of the completed, live system. Paste the most important result screenshots into the placeholders below.

6.1 GIKI Mart Storefront

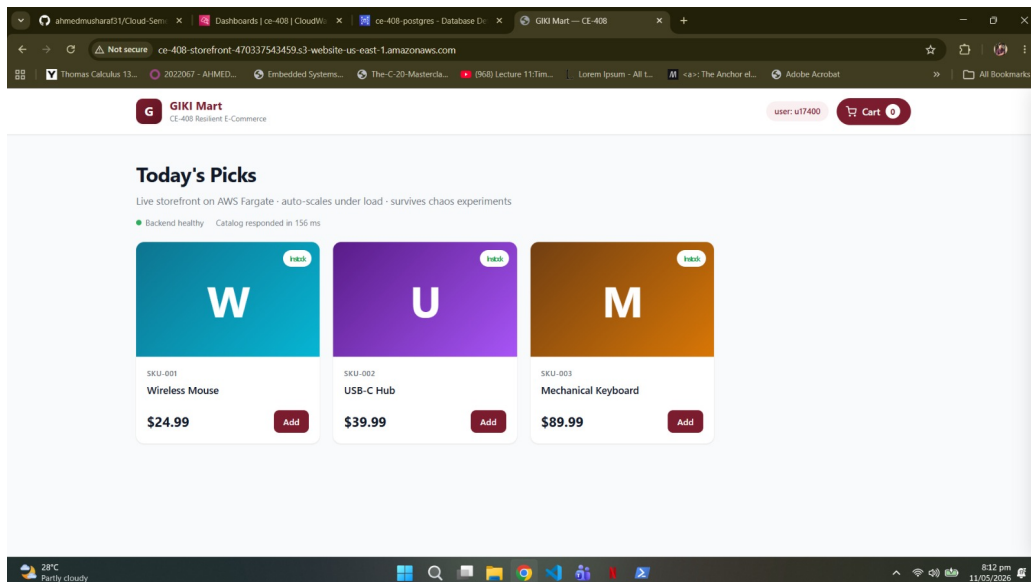


Figure 6.1: GIKI Mart storefront homepage: full product grid loaded from Fargate, showing catalog latency in the status bar and all three products in stock

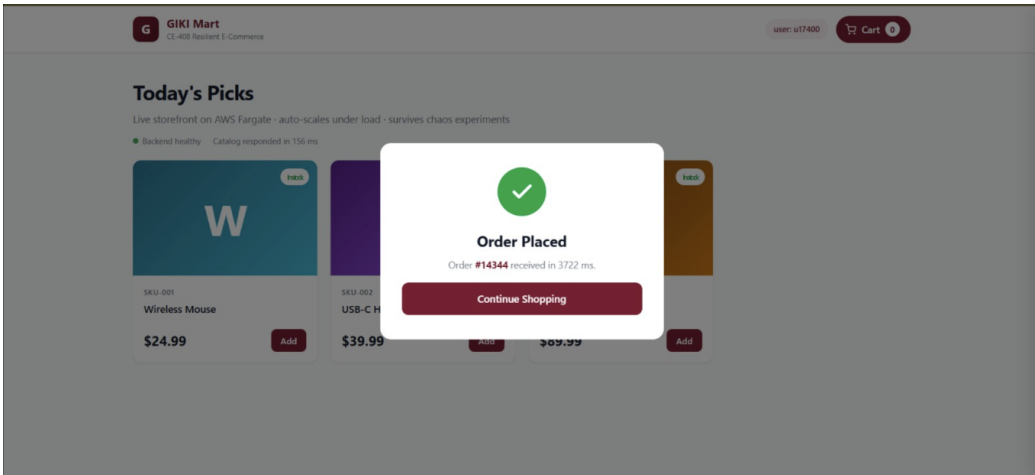


Figure 6.3: GIKI Mart order success modal: order ID and round-trip latency displayed after placing an order

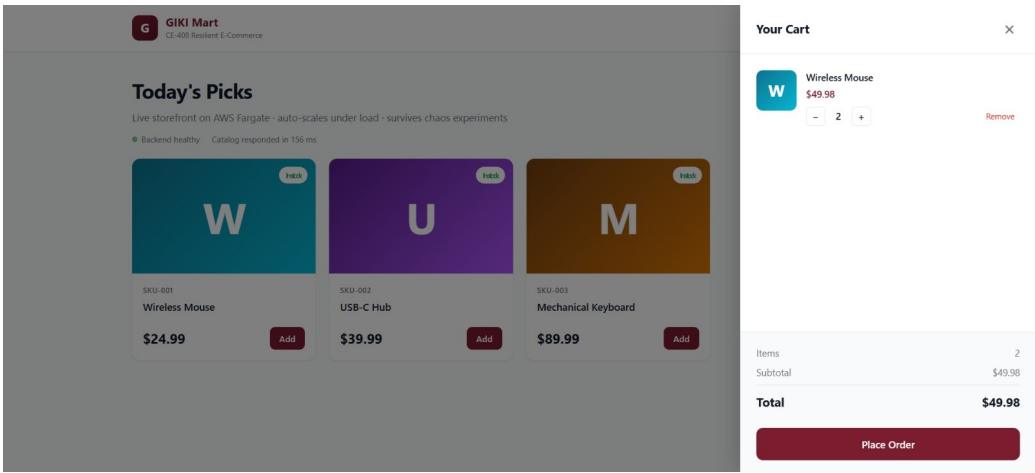


Figure 6.2: GIKI Mart cart drawer open: line items with quantity controls and total, ready for checkout

6.2 CloudWatch Dashboard — Live System

6.3 Load Test Results

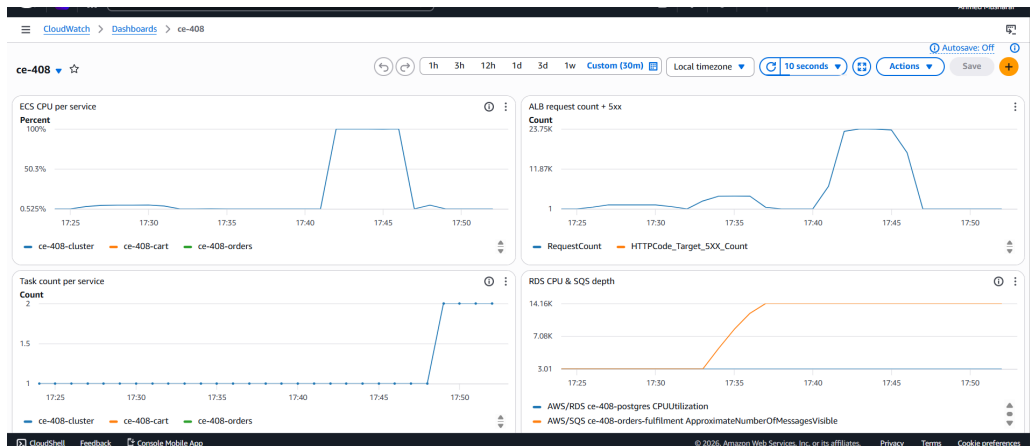


Figure 6.4: k6 console output for the stress test: VU ramp stages and final summary showing 0% errors and p95 within threshold

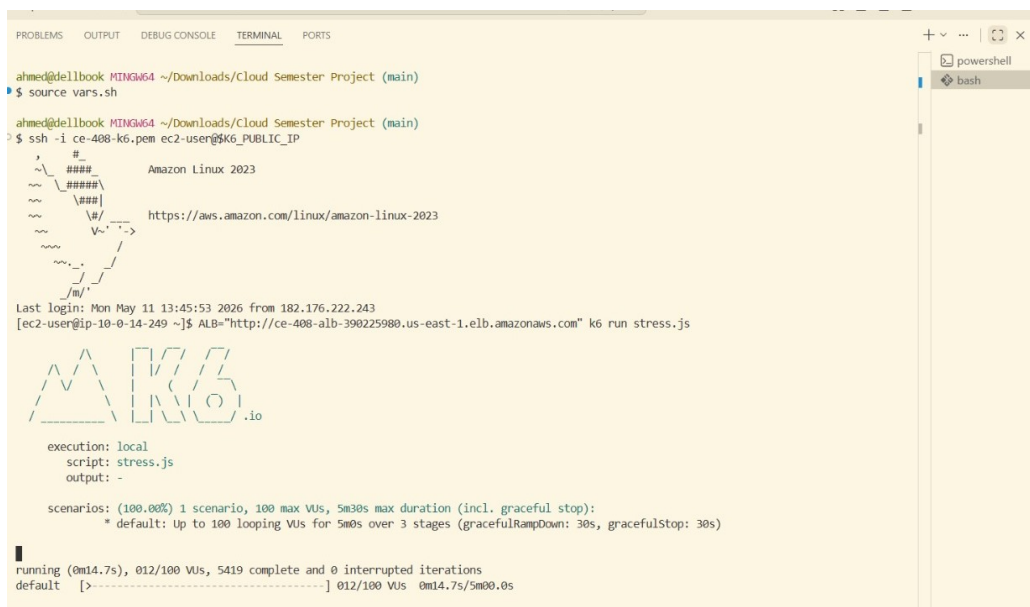


Figure 6.5: CloudWatch during stress test: Catalog CPU climbing above 60% and task count scaling from 1 to 2 tasks

Chapter 7

Discussion

7.1 Key Findings

Auto-scaling works as designed but has a detection lag. The CPU-target tracking policy requires approximately three consecutive minutes of CPU > 60% before an alarm fires, plus roughly two minutes for Fargate to provision a replacement task. The total detection-to-scale latency of 6–9 minutes is acceptable for sustained organic load growth but insufficient to absorb sudden, brief spikes. A step-scaling policy with a lower cooldown, or provisioned concurrency, would better handle flash-sale traffic patterns.

Task termination produces a short but bounded 5xx window. During Experiment 1, the 30-second ALB health-check interval means that after a task dies, the ALB continues forwarding some requests to the dead target until the next health check cycle. Reducing the health-check interval from 15 seconds to 5 seconds, combined with a shorter deregistration delay, would tighten the recovery window further.

Latency does not cascade across service boundaries. Experiment 2 confirms that the microservices architecture achieves failure isolation: injecting 500 ms of latency into Orders has zero measurable impact on Catalog or Cart response times, validating the design decision to avoid synchronous inter-service calls.

S3-hosted storefront is fully pause-safe. Unlike the ALB and NAT Gateway, the S3 static website incurs near-zero cost at all times and does not need to be torn down between sessions.

7.2 Challenges Encountered

- **CORS configuration** — All three FastAPI services initially lacked the `CORSMiddleware` middleware, causing the browser-based storefront to receive CORS errors when calling the ALB from the S3 origin. Adding the middleware and forcing a new ECS deployment resolved this.
- **Git Bash path mangling** — AWS CLI commands that include ARN strings beginning with / are misinterpreted by Git Bash on Windows as filesystem paths.

Prefixing commands with `MSYS_NO_PATHCONV=1` was required throughout the project.

- **SSM permissions** — The ECS task execution role needed an explicit `AmazonSSMReadOnlyAccess` attachment before containers could retrieve the DB password from Parameter Store; without it, task launches failed silently.
- **ALB path matching** — The listener rule `/orders/*` did not match `POST /orders` (no trailing path), requiring an additional exact-path rule at priority 31.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This project successfully demonstrates the construction and validation of a production-grade, cloud-native e-commerce backend on AWS. The three-service microservices architecture deployed on Fargate achieves independent horizontal scalability, polyglot persistence, and fault isolation. Auto-scaling is empirically validated to activate under sustained CPU pressure. Two chaos engineering experiments confirm that the system self-heals after task termination within 90 seconds and that latency faults do not propagate across service boundaries.

The accompanying GIKI Mart storefront provides a functional, browser-accessible demonstration of the complete stack, making the system's behaviour tangible and verifiable during live evaluation.

8.2 Future Work

1. **Amazon CloudFront + Route 53** — Serve the storefront from a CDN with a custom domain and TLS termination.
2. **Amazon Cognito** — Add user authentication to replace the random-guest user model.
3. **AWS X-Ray** — Enable distributed tracing across all three services to pinpoint latency contributors at the individual function level.
4. **ElastiCache for Redis** — Cache popular catalog queries to reduce RDS read pressure and lower p99 latency.
5. **AWS Fault Injection Service (FIS)** — Migrate the two chaos experiments to FIS for richer fault scenarios (AZ failure, CPU stress, network packet loss) and native integration with CloudWatch.
6. **Request-rate auto-scaling** — Supplement the CPU-target policy with ALB request-count target tracking to react faster to sudden spike traffic.
7. **CI/CD pipeline** — Add a GitHub Actions workflow to build, test, push images, and update ECS services automatically on every merge to **main**.

References

- [1] Amazon Web Services. *Amazon ECS Developer Guide*. <https://docs.aws.amazon.com/AmazonECS/latest/developerguide/>
- [2] Amazon Web Services. *Application Load Balancer User Guide*. <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/>
- [3] Amazon Web Services. *Amazon RDS User Guide for PostgreSQL*. <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/>
- [4] Amazon Web Services. *Application Auto Scaling Developer Guide*. <https://docs.aws.amazon.com/autoscaling/application/userguide/>
- [5] Grafana Labs. *k6 Documentation*. <https://k6.io/docs/>
- [6] Rosenthal, C., Jones, N. *Chaos Engineering: System Resiliency in Practice*. O'Reilly Media, 2020.
- [7] Fowler, M. *Microservices*. <https://martinfowler.com/articles/microservices.html>
- [8] Sebastián Ramírez. *FastAPI Documentation*. <https://fastapi.tiangolo.com/>
- [9] CE-408 Project Repository. <https://github.com/ahmedmusharaf31/Cloud-Semester-Project>